SamarthPlus — Incremental Timeline of Work

> Period covered (work owned by me): **Oct 2025 → Apr 2026** (≈ 7 calendar months) Primary contributor: Raunak Sharma (with collaboration from Vikas Singh and Yash on UI/SSO)

> **Scope note:** the initial Supabase prototype build (Aug → mid-Sep 2025) and the early stabilization work (mid-Sep) were done by others. This document starts at **Phase 1 = Supabase → Node.js migration (Oct 2025)** which is when I took over. References to earlier work appear only where they explain why something later was hard (e.g. business-name normalization debt carried over from the prototype).

The work below is laid out in the order it actually happened. Each phase says **what was built, what environment it ran in, why it took the time it did, and what bit us**. A consolidated module-share table sits at the bottom.

Environment progression at a glance

Phase	Window	Environment / user-base	Notes
1	Oct 2025	Dev only	Supabase → Node migration
2	Nov 2025	Dev only	SCORM/ILT/SSO/branding
3	Dec 2025 → Jan 2026	**Promoted to SIT**	Security hardening; SIT testing in parallel
4	Feb 2026	**UAT**	2FA, journey-scoping, real Reliance reviewers exercising the build
5	Mar 2026	UAT → pre-prod	Bulk upload tooling built specifically for the prod data lift
6	Apr 2026	**Prod (limited pilot)**	Massive prod-data load + pilot users

---

Phase 1 — Supabase → Node.js + Prisma migration (Oct 2025)

Stack before: Supabase (Postgres + Auth + Storage + Edge Functions). **Stack after:** Node.js + Express + Prisma + Postgres on Reliance infra, behind their gateway. **Decision driver:** Reliance ops required us off Supabase (data residency, vendor concentration, audit-trail control).

What we had to redo (in order)

1. **Backend project bootstrap** — Express + Prisma + TypeScript, env layout, logger, error middleware (`backend/src/`).
2. **Schema port** — every Supabase table + RLS converted into Prisma models + application-level authorization. RLS doesn't translate 1:1 — we read each policy and re-implemented as middleware/route guards.
3. **Edge functions → REST routes** — every Supabase edge function (admin user CRUD, OJT seed, proof upload, prescreening calc, certificate issue, eligibility refresh) re-implemented as Node services in `backend/src/routes/` and `backend/src/services/`.
4. **Storage migration** — Supabase Storage signed URLs replaced with our own signed-URL upload pipeline (`storage.ts`, `storageService.ts`).
5. **Auth re-build** — email-OTP, phone-OTP, JWT issuance, refresh handling — all of this lived in Supabase before; built from scratch in `email-auth.ts`, `phone-auth.ts`, `jwt-auth.ts`.
6. **Client integration** — every `supabase.from(...)` call in the React app swapped to our REST client. Done incrementally so the app stayed bootable on each merge.
7. **Pipeline + Docker** — Azure DevOps pipeline added, Dockerfile + entrypoint + compose for both frontend and backend.
8. **Prod data backfill** — `supabase-backup-20251014.tar.gz` was the source of truth; one-off script to land it in the new Postgres.

Why it took the full month and why difficult

- **Behavior parity, not schema parity.** Supabase auto-handles a lot (RLS, signed URLs, OTP, session). Each had to be re-thought, not mechanically copied. Auditing every read path for missing authorization was the biggest line item.
- **Two systems live in parallel.** The team kept shipping features in October, so Supabase had to keep working while Node caught up. Several merge cycles (`feat/node-migration-edge-functions` → `node-migration` → `main`) before cutover.
- **Prisma + Node 20 + bun pain.** Prisma engine version mismatch on Node 20, EOF errors, multiple rounds of entrypoint/config rewrites just to stabilize.
- **Storage cutover was risky.** Live SCORM packages, OJT proofs, certification PDFs sat in Supabase Storage. Had to copy + re-sign + verify nothing 404'd.
- **Pipeline was new.** Azure DevOps, Harbor registry, ADC credentials, Dockerfile flake-fixing — none of this existed before October.

Environment: Dev only. No external testers yet.

Phase 2 — SCORM, ILT, SSO, Reliance branding (Nov 2025)

The migration unlocked features that needed our own backend. **Still dev-only at this point.**

SCORM (the hardest sub-problem here)

1. **Chunked upload** — packages are 50MB-500MB zips; single multipart POST kept timing out behind the gateway. Built chunk upload with progress + resume.

2. **Manifest parsing** — `imsmanifest.xml` differs between SCORM 1.2 and 2004; some packages put title in `<organization>`, others in `<item>`. Title extraction broke initially.
3. **Asset path rewriting** — packages reference assets by relative path; we had to serve from blob storage with signed URLs while keeping in-package links working. This was the bulk of the player work.
4. **Player iframe sandboxing** — SCORM API needs to talk to parent (`window.API_1484_11`); had to bridge that without breaking CSP.
5. **Progress tracking back to backend** — `cmi.completion_status`, `cmi.success_status`, score wired to user_journey_progress.
6. **Validation at upload time** — manifest validation step so admins find out at upload time, not at play time.
7. *(Landscape orientation fix landed later in April 2026.)*

ILT (Instructor-Led Training)

- New tables (`20251110081254_add_instructor_led_training_tables`).
- Instructor dashboard (`ILTInstructorDashboard.tsx`), live-video circle on Screening + Learning.
- Disabled in user flow mid-month — OJT short-circuits past it until ILT data is ready.

SSO (Reliance PeopleFirst)

- `validateJwt` / `createJwt` exchange with PeopleFirst.
- Propagating BU code + role from PeopleFirst into our session.
- HRMS sync gated to SSO users only (no HRMS creds for non-SSO testers).

Reliance branding

- Logo swap everywhere, header image, favicon, certificate template.

Stabilization in parallel

- Public-video access fix; storage upload via signed URL (not direct).
- Auto-navigate flow: Self-paced → assessment → next step.
- DB-URL externalization, Prisma config rewrites for prod readiness.

Environment: Dev only.

Phase 3 — Security hardening + audit, promotion to SIT (Dec 2025 → Jan 2026)

Driven by **Fortify** and **BlackDuck** scan reports from Reliance security. **The build was promoted to SIT during this phase**, which meant Reliance's security + integration testers were scanning and exercising the app while we were still hardening it — every fix had to land without breaking the SIT users.

What shipped, in roughly the order it landed

- **CSP headers** + same-site cookies (Dec).

- **Audit trail** — extensive logging service so we can prove who-did-what (compliance ask).
- **Secure encryption** for sensitive payloads + tokens; replaced legacy hashes/AES modes (end of Dec).
- **Session-based auth** — single-session enforcement via `user_sessions` table; conflict warning if user signs in elsewhere. Took the longest in this phase because session conflict needed both backend invalidation + graceful frontend handling.
- **Revoked-tokens table** (`20251203011238_add_revoked_tokens_table`).
- **Input validation** — server-side schemas added across every form (was previously mostly client-side).
- **File-upload restrictions** — extension whitelist, MIME re-check on server, size cap, media-only routes.
- **Auth hardening** — max-login-attempt lockout (`login_attempts` table, mid-Jan), refresh-token expiry shortened.
- **XSS fixes** — sanitized every render path that took user input, replaced dangerous innerHTML.
- **All Fortify Critical + High items** cleared (3 critical + 2 high in one batch, then a second pass for the rest).
- **BlackDuck dependency upgrades.**
- **SSL cert** refresh, URL validation tightening.
- **2FA scaffolding** added late Jan — login changes prepared for 2FA, then **reverted** because the UX wasn't right; properly re-implemented in Phase 4.

Why this phase was so long

- Fortify reports listed **hundreds of findings**; each Critical needed code-level fix + verification + re-scan loop.
- Re-scan turnaround was substantial, so the rate-limit was scan cycles, not coding speed.
- Several fixes (CSP especially) broke functionality in non-obvious ways — e.g. SCORM iframe stopped working when CSP was tightened, had to refine policy iteratively.
- **SIT testers were filing tickets in parallel** — bugs surfaced by their flows had to be triaged alongside the security work.

Environment: SIT. First time the app was being used by people outside the team.

Phase 4 — 2FA + journey-scoping + stabilization, promotion to UAT (Feb 2026)

Build was promoted to UAT this month. Real Reliance business reviewers (HR + L&D) were exercising the app on actual journeys, which is when most of the data-shape and journey-scoping bugs surfaced.

2FA (proper rollout this time)

- New `auth_2fa_challenges` table.
- Email-OTP-based 2FA after primary login.
- Boxed OTP input, 60s expiry, resend button, timer alignment — UI iterated against UAT feedback.

Journey-scoping (the biggest non-2FA work this month)

Originally OJT, prescreening, learning all lived globally per user. Made everything **scoped to the active journey** so a user mid-journey-A wouldn't see journey-B's OJT pending or cross-pollinate progress.

Migrations: `20260219_add_journey_id_to_ojt_instances`, `20260221155000_add_journey_id`, `20260313_add_ojt_user_journey_unique`.

This was painful because it touched **every read path** in the app — UAT users hit the bugs first (intermittent OJT 500s, prescreening reset issues, video progress not reflecting), so the cycle was: UAT report → reproduce → fix → redeploy to UAT.

Stabilization fixes (largely UAT-driven)

- OJT dashboard / creation / blocking issues (caused by stale journey context).
- Video completion + progress not reflecting after refresh.
- Intermittent logout on OJT 500 errors (was treating any 500 as auth failure).
- Certification PDF — invalid date, rendering, download path.
- SSO landing page after login.
- Demo-seen mandatory before OJT activity start.
- Career-tree active + completed journey overlay (career tree finally feels stable, though still not clean — that waits for April).
- HRMS sync for current BU code + tenure days (`current_role_tenure_days` migration).
- XSS handling for residual paths.

Environment: UAT. Active feedback loop with Reliance reviewers.

Phase 5 — Bulk upload + assessment retake + prescreening polish (Mar 2026)

Still on UAT, but the focus shifted: we knew prod cutover was near, and prod would need a **massive amount of configuration data** loaded (model stores, eligibility, skills, OJT applicability matrix, BU mappings). The bulk-upload tooling was built specifically for that lift.

Bulk upload (main effort)

- Excel template upload for users, **model stores**, addresses, eligibility requirements.
- Plain-text fallback when admin pastes data instead of using the template.
- Optimization pass mid-March — large sheets (10k+ rows) were timing out; chunked processing + better error reporting.
- Per-row error report so admins fix and re-upload rejected rows only.

Assessment retake matrix

- Second-attempt logic, rebound time after failure.
- Retake unique-index migration (`20260310_fix_skill_attempt_unique_index`).

Prescreening enhancements

- More info screen, prescreening attempts persistence, prescreening tests config.

SCORM

- Title parse fix, JS-file upload error, manifest fix.

Environment: UAT, with prod cutover preparation in parallel.

Phase 6 — Prod cutover, mass data upload, pilot testing on prod (Apr 2026)

Build went to Prod this phase, with a limited set of pilot users. This phase was as much about **loading the prod configuration data** as it was about code.

Prod data uploaded (the largest non-code effort of the project)

- **Model stores** — entire Reliance store master, region- and BU-tagged.
- **Business Unit master** — backed by the new `20260428_add_business_unit_master` migration.
- **Skills + applicability matrix** — per BU, per role.
- **Eligibility criteria** — per role, including activated-certification rules.
- **OJT applicability matrix + checklist library** — `20260415_seed_ojt_grocery_fl` plus admin-driven uploads.
- **User base** — initial pilot users, addresses, role/BU mapping.
- Multiple rounds of re-upload because source Excel sheets had inconsistent BU names / role codes (this is the same normalization debt we'd been carrying since the prototype, finally cleaned up via the BU master).

Code that landed alongside the data

- **Business Unit master table** (`20260428_add_business_unit_master`) + `business-units.ts` route — driving model-store and store-mapping bulk update via **BU code** instead of free-text business name. **This finally fixes the business-name-normalization problem the project had been patching for months.**
- **Eligibility checks** updated for activated certifications + dynamic requirement lookup.
- **State filter** in admin views.
- **SCORM landscape orientation** fix (tablets defaulted to portrait; player was clipping).
- **Deploy fixes** — Sendgrid env vars, Docker pipeline failure, main DB URL plumbing through entrypoint.
- Prod migrations finalized.

Pilot testing on prod

- Limited pilot user set running real journeys.
- Issues reported are now graded by severity (blocker / non-blocker for wider rollout).
- Sendgrid mail delivery, SSO end-to-end, certificate issuance verified on real prod traffic.

Environment: Prod (limited pilot).

Cross-cutting challenges (the things that recurred across phases)

1. **Populating prod data was the single biggest non-coding cost** — formalized in Phase 6 but felt across every phase.
   - Career Tree, OJT applicability matrix, skills, eligibility criteria, model stores — every one needed real Reliance data.
   - Source data arrived as Excel sheets with inconsistent BU names, missing role codes, K-level vs role-name mismatches. We wrote one-off scripts repeatedly; the **April 2026 BU master + bulk upload finally formalized this** and the prod data load happened cleanly.
2. **Career Tree, specifically** — three independent sources of truth for "user's current business and role" (HRMS, user profile, journey config). Every fix to one surfaced a mismatch with another. Took until **Feb 2026** (active + completed journey overlay) to feel stable; took until **Apr 2026** (BU master) to be clean.
3. **SCORM packages** — third-party packages don't follow a single spec strictly. Even after our parser handled SCORM 1.2 + 2004, individual packages had quirks (custom manifest extensions, broken relative paths, missing resources). Each new vendor package surfaced a new edge case.
4. **Migration parallel-run** — keeping Supabase alive while Node matured added overhead in Oct (every feature shipped twice).
5. **Fortify scan loop** — long turnaround per scan cycle; many cycles needed to clear the queue.
6. **Journey context propagation** — making everything journey-scoped retroactively touched every read path. Bugs leaked through to UAT for weeks.
7. **Reliance infra constraints** — Harbor registry, ADC credentials, restricted egress, SSL cert rotation, Sendgrid setup all needed coordination with Reliance ops, not pure dev work.
8. **Environment promotions had cost** — every time we moved Dev → SIT → UAT → Prod we had to redo env-var plumbing, DB-URL handling, certificate trust, Sendgrid keys, and hand off to a different tester group.

Module share by phase (relative weight, not absolute time)

Module	Heaviest phase(s)
Backend migration (Supabase→Node)	Phase 1 (Oct)
SCORM	Phase 2 (Nov), Phase 5 (Mar), Phase 6 (Apr)
ILT	Phase 2 (Nov)
SSO (PeopleFirst)	Phase 2 (Nov), Phase 4 (Feb)
Reliance branding	Phase 2 (Nov)
Security hardening (Fortify/BlackDuck)	Phase 3 (Dec → Jan)
Sessions / auth hardening	Phase 3 (Dec → Jan)
2FA	Phase 3 (Jan, scaffold) → Phase 4 (Feb, ship)
Journey-scoping (OJT/prescreening/learning)	Phase 4 (Feb)
Certifications	Phase 4 (Feb)
HRMS integration	Phase 4 (Feb)
Bulk upload tooling	Phase 5 (Mar)
Assessments (retake matrix)	Phase 5 (Mar)
Pre-Screening enhancements	Phase 5 (Mar)
Prod data upload (config + users)	Phase 6 (Apr)
BU master / business-name normalization	Phase 6 (Apr)
Eligibility checks	Phase 6 (Apr)
OJT (continuous improvement)	Phases 2-6 (Nov → Apr)
Admin Dashboard & User Management	Phases 2-6 (Nov → Apr)
DevOps / Deployment / Pipelines	Phases 1, 3, 6 (env promotions)
Testing	Phases 1-2 (Oct → Nov)

Notes on the methodology

- The doc tracks **calendar months and phases**, not hours or person-days.
- Where a single phase touched multiple modules, the module-share table indicates relative weight rather than precise allocation.
- Excludes: design discussions, manual QA, stakeholder meetings, environment setup not visible in source control.
- Earlier prototype work (Aug → mid-Sep 2025) is not counted here as it was done by others; it appears only in cross-cutting context (e.g. inherited business-name normalization debt).